02613 Python and High-Performance Computing

Exam 2024

Written examination date: 28.05.2024

Page 1 of 14 pages

Course name: Python and High-Performance Computing

Course number: 02613

Aids allowed: Written works of reference are permitted

Exam duration: 4 hours

Weighting: The grade is determined based on an overall assessment of all answers to the exam questions.

For all *non-multiple choice* problems, when answering the different questions, remember to show/explain your calculations and motivate your answers. If you make additional assumptions, remember to briefly explain them. Failure to do so may cause substantial reduction of the number of points given. You may write your answers directly on the exam set, on separate papers, or both. For answers on separate papers, remember to clearly indicate what question your are answering.

For all *multiple choice* problems you may simply mark your chosen answer with no additional justification. If you wish to change your answer, cross out your current mark and write your chosen option next to the question.

Unless otherwise specified:

- All arrays are NumPy arrays and stored row-wise.
- `np` refers to the NumPy Python module.

You are a performance consultant and have been sent by DTU to help out a company called Synergistic Logistics Operational Technologies Hub. They are doing many different kinds of data processing, all of which are currently running too slow.

One of the engineers is keen to move some processing to batch jobs. They show you a job script they are working on. They are having issues with the resource specifications. The job script is:

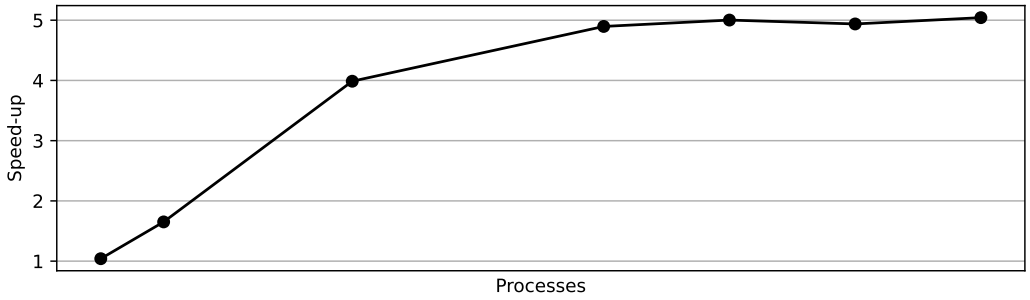
```
#!/bin/bash
#BSUB -J proc
#BSUB -q hpc
#BSUB -W 00:05
#BSUB -R "rusage[mem=4GB]"
#BSUB -n 1
#BSUB -o proc_%J.out
#BSUB -e proc_%J.err

python process.py allthe.data
```

They explains that the program usually runs best with 8 cores. It uses 16GB of memory and takes at most 15 minutes to complete.

1. How would you change the job script above to accommodate this? Use only the resources you need.

   Change the following options:
   `#BSUB -W 00:15`
   `#BSUB -n 8`
   `#BSUB -R "rusage[mem=2GB]"`
   `#BSUB -R "span[hosts=1]"`

Pleased with your solution, another engineer asks for help with a different program. They are considering to use multi-processing in order to use less time. A previous engineer did some measurements and created the speed-up plot below. Sadly, they forgot to mark how many processes were used for each run.



2. Given the above speed-up plot, what is the parallel fraction of the program?
   (a) 0.2 (b) 0.5 (c) 0.8 (d) 0.9

   (c) 0.8

Management wants at least a speed-up of 4, before they consider parallelization to be worth it. Currently, the most powerful machine at the company has 8 cores.

3. Given your estimated parallel fraction, should they pursue parallelization? Explain your reasoning:

   Parallel fraction $F = 0.8$. Speed-up with 8 cores according to Amdahl's law: $S(8) = \frac{1}{1-F+F/8} = \frac{1}{\frac{1}{5}+\frac{1}{10}} = \frac{10}{3} < 4$. No, they should *not* pursue parallelization because they won't get a speed-up of 4 with their most powerful machine.

Excited about all the parallelization discussions, another engineer comes by with a different problem. They are trying to compute the value of the following function for each value of an array `numbers` $= [1, 2, 3, \ldots, 100\,000\,000]$. The function is:

```python
def process_number(n):
    s = 0
    for i in range(n):
        s += i
    return s
```

To speed this up, they want to use parallelization to spread the work out to several workers.

4. What Python parallelization approach should they use for this function?
   (a) Multi-threading (b) Multi-processing (c) Does not matter

   (b) Multi-processing

5. Should they use static scheduling or dynamic scheduling for this task? Explain your answer:

   The time it takes to process each number will vary a lot. Therefore, they should use *dynamic scheduling* so one process does not finish much faster than the others.

Impressed by your extensive knowledge of parallelization, yet another engineer approaches you with a problem. They are trying to compute the absolute value of a sum, i.e., $|x_1 + x_2 + \cdots + x_n|$. To speed this up, they want to use a parallel reduction and have made the following function to compute each partial result:

```python
def abssum(x, y):
        return abs(x + y)  # Compute |x + y|
```

However, they are having trouble getting the correct answers consistently.

6. Explain why this function *cannot* be used with the parallel reduction tree framework. What should the engineer do instead?

   The function is not associative: $||1+2|+(-3)| = 0 \neq |1+|2+(-3)|| = 2$. Therefore, it can't be used in a parallel reduction. Instead, they should just do a normal parallel sum-reduction and take the absolute value at the end.

Eager to make further use of your knowledge, management sends you to another department. Here, an engineer is working with a collection of RGB images stored as an $N \times H \times W \times 3$ array named `images` where N is the number of images, H is the image height, W is the image width, and the last axis represent the RGB channels. They want to subtract a mean image `mim` stored as an $H \times W$ array from each color channel in every image in `images`.

7. Which of the following NumPy operations will achieve this?
   (a) `images - mim`
   (b) `images - mim[:, :, None]`
   (c) `images - mim[None]`

   (b) `images - mim[:, :, None]`

Another engineer in the department is writing a function to process the pixel values. To speed up their code, they are using Numba. Their function looks like this:

```
@jit(nopython=True)
def process(images):
    for i in range(images.shape[0]):
        for j in range(images.shape[1]):
            for k in range(images.shape[2]):
                for l in range(images.shape[3]):
                    x = images[i, j, k, l]
                    ...
```

However, they are having trouble getting the performance they want. They tried reversing the loop order, but that did not help either. You decide to inspect the strides of the `images` array and get `(600, 40, 8, 200)`.

8. Based on this information, how would you re-order the loops in the `process` function? Explain your answer:

   We order the loop so the axis with the shortest stride is inner-most. This means we fully use each cache-line so the code is cache-efficient. From inner-most to outer-most: `k, j, l, i`.

You are now sent to the statistics department. They have recently developed a fancy new model and are eager to apply it. However, it is currently running too slow. Their Python program that does the data processing looks like this:

```
1  import ourlib
2
3  samples = ourlib.load_data('data.data')
4  parameters = ourlib.load_params('params.data')
5  model = ourlib.prepare_model(parameters)
6  results = []
7  for s in samples:
8      r = ourlib.process_sample(model, s)
9      results.append(r)
10
11 ourlib.save(results, 'results.txt')
```

The normal workload is to process at least 1 000 samples at a time. For profiling, they have prepared a smaller data subset. The output of the profiler is:

```
        238 function calls (237 primitive calls) in 44.126 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   2/1    0.000    0.000   44.126   44.126 {built-in method builtins.exec}
     1    0.000    0.000   44.126   44.126 process.py:1(<module>)
     1    0.000    0.000   20.009   20.009 ourlib.py:3(load_params)
     1    0.000    0.000   15.013   15.013 ourlib.py:8(prepare_model)
    10    0.000    0.000    5.055    0.505 ourlib.py:13(process_sample)
     1    0.000    0.000    3.007    3.007 ourlib.py:18(save)
     1    0.000    0.000    1.001    1.001 ourlib.py:26(load_data)
     1    0.000    0.000    0.042    0.042 <frozen importlib._bootstrap>:97...
     1    0.000    0.000    0.042    0.042 <frozen importlib._bootstrap>:94...
     1    0.000    0.000    0.041    0.041 <frozen importlib._bootstrap>:66...
     ...
```

9. How many samples were in the data subset?
   (a) 2 (b) 5 (c) 10

   (c) 10

10. On what function would you focus your optimization efforts w.r.t. normal work loads? Explain your answer:

    For normal work loads, the `process_sample` function would use $1000 \cdot 0.505 = 505$ seconds. This is much more than the other functions, so we should focus on `process_sample`.

    One can argue that `load_data` and `save` may also scale with more samples but that is hard to judge from the given information, whereas `process_sample` is sure to dominate.

They now transfer you to the machine learning department. Here, they are processing multi-spectral images where each image is of size $h \times w$ and each pixel stores c channels, i.e., each pixel is a c-dimensional vector. As part of their processing pipeline, they need a function that convolves each pixel with a length c kernel. They have developed a parallel function for this, which, in pseudo-code, works like this:

```
def conv_channels(image, kernel, out, h, w, c):
    # Process each pixel in parallel
    parallel for i in range(h):
        parallel for j in range(w):
            o = 0.0
            # Loop over channels for current pixel
            for k in range(c):
                o += image[???] * kernel[k]
            out[i, j] = o
```

They are discussing how to store the `image` array to maximize cache efficiency. The options are to store the images with the channels as the first dimension (i.e., `image` has shape $c \times h \times w$) or the channels as the last dimension (i.e., `image` has shape $h \times w \times c$).

11. Given the above pseudo-code, how should the `image` array be stored to maximize cache efficiency? Explain your answer:

    It should be stored with channels as the *last* dimension. The inner loop executed by each thread runs over the channels, therefore this axis should have the shortest stride to mimimize cache loads. As we store the array row-wise, the last axis has the shortest stride.

To further increase the performance, they are exploring GPU processing with CUDA. They have created the CUDA kernel shown below to perform the same processing.

```
@cuda.jit
def conv_channels_kernel(image, kernel, out, h, w, c):
    i, j = cuda.grid(2)
    if i < h and j < w:
        o = 0.0
        for k in range(c):
            o += image[???] * kernel[k]
        out[i, j] = o
```

12. Given the above implementation, how should the `image` array be stored to maximize cache efficiency? Assume the kernel is called with thread blocks of size 32×32. Explain your answer:

    It should be stored with the channels as *first* dimension. Each thread in a thread block will access the values for channel `k` at the same time. Since threads in a block share the same cache, the values for each channel should be close in memory, i.e., the spatial axes must be last. Also, threads within the same warp can share cache line loads, so the threads in a warp should ideally be aligned with the smallest stride.

To understand the performance of the GPU implementation better, they run it through the nsys profiler. The relevant parts of the result are shown below:

```
** CUDA GPU Kernel Summary (gpukernsum):

Time (%)  Total Time (sec)  Instances  Avg (sec) ... Name
--------  ----------------  ---------  --------- ... ----------------------------...
   100.0            0.5000          1     0.5000 ... cudapy::__main__::conv_chan...

** GPU MemOps Summary (by Time) (gpumemtimesum):

Time (%)  Total Time (sec)  Count  Avg (sec)  Med (sec) ...     Operation
--------  ----------------  -----  ---------  --------- ... ------------------
    83.3            2.5000      2     1.2500     1.2500 ... [CUDA memcpy HtoD]
    16.7            0.5000      1     0.5000     0.5000 ... [CUDA memcpy DtoH]

** GPU MemOps Summary (by Size) (gpumemsizesum):

Total (MB)  Count  Avg (MB)   Med (MB)   Min (MB)  Max (MB)  ...     Operation
----------  -----  ---------  ---------  --------  --------- ... ------------------
 25000.000      2  12500.000  12500.000     0.000  25000.000 ... [CUDA memcpy HtoD]
  1000.000      1   1000.000   1000.000  1000.000   1000.000 ... [CUDA memcpy DtoH]
```

13. Given the above profiler output, what is the estimated transfer speed from CPU memory to GPU memory?
    (a) Around 2 GB/s (b) Around 10 GB/s (c) Around 12.5 GB/s

    (b) Around 10 GB/.

An engineer informs you that the current pipeline which uses the parallel CPU version can compute the results for the same inputs in 7 seconds.

14. Keeping the rest of the pipeline the same, how much faster would it be to use the CUDA version `conv_channels_kernel` compared to the parallel CPU version `conv_channels`?

    Total time for the GPU version is kernel time plus transfer times: $0.5 + 2.5 + 0.5 = 3.5$. This is 2x faster than the CPU version.

The machine learning department thanks you for your great advice and sends you on to the business analytics department. They are working with a large data frame of factory productivity data. Below is a summary of the data frame that shows the name, data type, number of unique values, minimum value, maximum value, an example value and memory use of each column:

col. name	dtype	#unique	min	max	example	size
`date`	`object`	70 079	1829-09-01	2024-05-28	2024-05-28	7.98 GB
`location`	`object`	8	N/A	N/A	Lyngby	7.64 GB
`mach_id`	`int64`	5 731	-1	5 730	1 234	1.07 GB
`units`	`int64`	43 923	932	68 837	1 234	1.07 GB

15. Briefly explain how and why/why not you would recode each column of the data frame in order to reduce the memory footprint:

    The `date` column is currently stored as strings which wastes space. I would recode it to a `datetime` data type. Recoding to an integer, e.g., number of days since the minimum date would be acceptable with arguments.
    The `location` column is also stored as strings. Since there is only 8 unique values, I would recode it to a categorical value, ideally with a `uint8` index. Recoding to an 8-bit integer would also be okay.
    The `mach_id` column could be converted to an `int16` data types since the minimum and maximum values fit in the range of `int16`.
    The `units` column could be converted to a `uint32` or `int32` since the minimum and maximum values fit in the range of 32 bit integers.

The most common operation they perform with the data frame is to extract the rows corresponding to a certain day and then compute some summary statistics. They typically conduct many such operations every time the data frame is used. However, these operations are currently running too slow.

16. Briefly explain how they may speed up such operations:

    They can set the `date` column as the index and sort it. This will speed up queries based on the date by a lot. Since they do many such queries, the overhead for building the index will be worth it.

They are also dealing with another problem. Here, they have a set of data frames that each hold power measurements from a large group of sensors. The sensors perform measurements every second, and each data frame stores a month of data for all sensors. A summary of a typical data frame is shown below:

col. name	dtype	#unique	min	max	size
`sensor_id`	`uint32`	11 073	8 031	124 282	1.91 GB
`timestamp`	`uint64`	2 505 600	1 706 742 000	1 716 847 200	3.83 GB
`power`	`float64`	1 825 832	0.000	48.217	3.83 GB

They have already reduced the memory use of each column as much as possible. However, it is still too large. They therefore want to process the data frame in chunks, in order to further reduce the memory use. The hardware they want to use only has 200 MB of available memory for holding the data.

17. What is the maximum amount of rows that can fit in each data frame chunk?
    (a) Around 10 000 (b) Around 10 000 000 (c) Around 10 000 000 000

    (b) Around 10 000 000

To process data frames from several months as fast as possible, they use a job array via the following job script:

```
#!/bin/bash
#BSUB -J power[1-12]
#BSUB -q hpc
#BSUB -W 02:30
#BSUB -R "rusage[mem=250MB]"
#BSUB -n 1
#BSUB -o power_%I_%J.out
#BSUB -e power_%I_%J.err

python poweranalysis.py $LSB_JOBINDEX data/power_measurements
```

They have *another* job that needs to run once all jobs in the above job array have finished processing. It does not matter if each individual job finished successfully or not—as long as it finished.

18. What must they add to their job script in order to achieve this?

    Add the line: `#BSUB -w ended(power)` to wait until all jobs in the `power` job array has finished with state `EXIT` or `DONE`.

Finally, you are sent to the numerical simulations department. They are currently trying to simulate the behavior of a dynamical system given a range of initial values. Specifically, they are working with the code below:

```
@jit(nopython=True, nogil=True)  # nogil=True --> Release GIL in this function
def simulate_single(x0, n, step):
  x = x0
  for i in range(n):
      x = x + step * np.cos(x) / (np.sin(5*x) + 2.5)
  return x
```

```
7
8  def simulate(n, x0s, step):
9      m = len(x0s)
10     out = []
11     for j in range(m):
12         r = simulate_single(x0s[j], n, step)
13         out.append(r)
14     return out
```

The code in `simulate_single` performs `n` steps of simulation given an initial value `x0`. The code in `simulate` then calls the `simulate_single` functions with `m` different initial values. To speed up their simulation, they would like to apply parallelization.

19. How and where would you apply parallelization in the above code? To what extent? What is your expected speed-up? How does your advice depend on `m` and `n`? You may ignore overhead in your analysis.

    I would apply parallelization over the loop in `simulate`. I can't do it over the loop in `simulate_single` since each step depends on the previous. I would use multi-threading, since `simulate_single` releases the GIL. I would apply up to `m` threads which should give me speed-up of `m` times, ignoring overhead. My speed-up does not depend on `n`.

End of exam